Lab#1



– Digital Electronics –

Combinational Circuits

The purpose of this laboratory session is to learn how to connect simple input and output devices to an FPGA chip and implement a combinational circuit that uses these devices for a specific target. We will use the switches *SW*9-0 on the DE1-SoC board as inputs and we will use light emitting diodes (LEDs) and 7-segments displays as outputs of our logic circuit.

Contents:

1. Controlling the LEDs
2. 2-to-1 Multiplexer
3. Controlling a 7-segment displeay
4. Comparing two values

Abbreviations and acronyms:

IC – Integrated Circuit
LED – Light Emitting Diode
MUX – Multiplexer

USE EXACTLY THE SAME I/O PINS SPECIFIED IN THIS DOCUMENT.

1 – Controlling the LEDs

The DE1-SoC board provides 10 toggle switches, called *SW*9-0 that can be used as circuit inputs, and 10 red lights, called *LEDR*9-0, that can be used to display output values. Figure 1 shows a simple Verilog module/VHDL entity that uses these switches and shows their states on the LEDs. As specified in the code, since there are 10 switches and 10 LEDs, it is convenient to represent them as arrays. Here we have used a single assignment statement for all 10 *LEDR* outputs, which is equivalent to the following individual assignments:

```
// Verilog
assign LEDR[9] = SW[9];
assign LEDR[8] = SW[8];
. . .
assign LEDR[0] = SW[0];

-- VHDL
LEDR(9) <= SW(9);
LEDR(8) <= SW(8);
. . .
LEDR(0) <= SW(0);
```

The DE1-SoC board has hardwired connections between the FPGA chip, the switches and the LEDs. To use *SW*9-0 and *LEDR*9-0 it is necessary to include in your Quartus II project the correct pin assignments, which are given in the *DE1_SOC_User_ Manual*. For example, the manual specifies that *SW*0 is connected to the FPGA pin *PIN_AB12* and *LEDR*0 is connected to pin *PIN_V16*. A good way to make the required pin assignments is to import into the Quartus II software the file called DE1_SoC.qsf, which is provided on the web site of the course. The procedure for making a pin assignment is described in the first lab session LAB#0…

It is important to realize that the pin assignments in the DE1_SoC.qsf file are useful only if the pin names given in the file are exactly the same as the port names used in your HDL module/entity. The file uses the names *SW*[0] . . . *SW*[9] and *LEDR*[0] . . . *LEDR*[9] for the switches and lights. This is the reason why we used these names in Figure 1 (note that the Quartus Prime software uses [ ] square brackets for array elements, while the VHDL syntax uses ( ) round brackets).

```
// Verilog
module part1 (
    input  9:0] SW,     // Input switches
    output [9:0] LEDR    // Output LEDs
);

    assign LEDR = SW;         // Connect all switches directly to LEDs
endmodule

-- VHDL
LIBRARY ieee;
USE ieee.std_logic_1164.all;
-- Simple module that connects
-- the SW switches to the LEDR lights

ENTITY part1 IS
      PORT ( SW  : IN STD_LOGIC_VECTOR(9 DOWNTO 0);
      LEDR : OUT STD_LOGIC_VECTOR(9 DOWNTO 0)); -- red LEDs
END part1;

ARCHITECTURE Behavior OF part1 IS
BEGIN
      LEDR <= SW;
```

```
    END Behavior;
```

Figure 1 – Verilog/VHDL code that uses the switches and LEDs on the DE1 board.

You need to do steps and implement a circuit corresponding to the code in Figure 1 on the DE1 board.

1. **Create a new Quartus II project** for your circuit. Select Cyclone V 5CSEMA5F31C6 as the target chip, which is the FPGA chip on the Altera DE1 board.
2. **Create a Verilog module/VHDL entity** for the code in Figure 1 and include it in your project.
3. Include in your project the required **pin assignments** for the DE1 board, as discussed above.
4. **Compile** the project.
5. **Download** the compiled circuit in the FPGA chip.
6. **Test** the functionality of the circuit by toggling the switches and looking at the LEDs.

PLEASE REFER TO THE LAB#0 FILE FOR DETAILS ON PROGRAMMING THE FPGA

2 - 2-to-1 Multiplexer

Figure 2 shows a 2-to-1 *multiplexer* with a select input *s*.

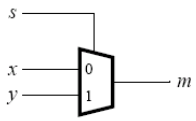


Figure 2 - A 2-to-1 multiplexer.

The multiplexer can be described in Verilog or in VHDL with the following statements:

// Verilog

```
assign m = (s == 1'b0) ? x : y;
```

-- VHDL

```
m <= x when (s = '0') else y;
```

You need to write a Verilog module or VHDL entity that includes the statement shown above to describe the circuit given in Figure 3. This circuit has two four-bit inputs, *X* and *Y*, and produces the four-bit output *M*. We refer to this circuit as a four-bit wide 2-to-1 multiplexer. The signals *X*, *Y* and *M* are depicted as four-bit wires. You need to do the steps shown below.

1. **Create a new Quartus Prime project** for your circuit.
2. **Write and include your Verilog/VHDL file** for the four-bit wide 2-to-1 multiplexer in your project. Use switch *SW* 8 on the DE1-Soc board as the *S* input, switches *SW*3-0 as the *X* input and *SW*7-4 as the *Y* input. Connect the output *M* to the red lights *LEDR*3-0.
3. Include in your project the required **pin assignments** for the DE1-Soc board. As discussed in Part I, these assignments ensure that the input ports of your HDL code use the pins on the Cyclone V FPGA that are connected to the *SW* switches. Moreover, ensure that the output ports of your HDL code use the FPGA pins connected to the *LEDR* lights.
4. **Compile** the project.
5. **Download** the compiled circuit into the FPGA chip.
6. **Test** the functionality of the four-bit wide 2-to-1 multiplexer by toggling the switches and observing the LEDs.

1. **YOU NEED TO WRITE THE HDL CODE OF THE 4 BIT 2-TO-1 MULTIPLEXER.**
2. **YOU NEED TO TEST THE MULTIPLEXER ON THE FPGA.**

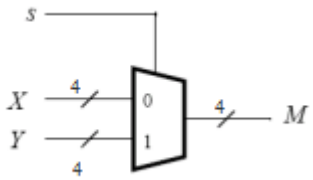


Figure 3 - A four-bit wide 2-to-1 multiplexer.

3 - Controlling a 7-segment display

Figure 4 shows a *7-segment* decoder module with a 4-bit input **c**. This decoder drives seven outputs that are used to display an alphanumeric character on a 7-segment display. Table 1 lists the characters that should be displayed for each value of **c**.

The seven segments in the display are identified by the indexes 0 to 6 shown in figure. Each segment is lit when driven to the logic value '0'. You need to write a Verilog module/VHDL entity that implements the seven segments decoder. Use an always or process with a case or case-when statement. For VHDL remember to use the *numeric_std* package from the *ieee* library in order to have all the required functions.

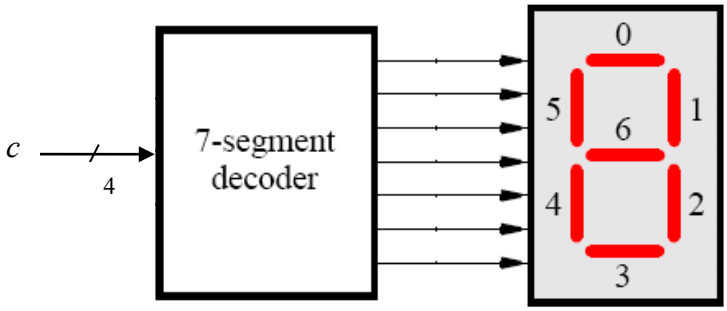


Figure 4 - A 7-segment decoder.

c	Character
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5

0110	6
0111	7
1000	8
1001	9
1010	A
1011	b
1100	C
1101	d
1110	E
1111	F

Table 1 - Character codes.

In particular, you need to do the following steps:

1. **Create a new Quartus Prime project** for your circuit.
2. **Create a Verilog module or VHDL entity** for the 7-segment decoder. Connect ***c*** to switches *SW*3-0, and connect the outputs of the decoder to the *HEX0* display on the DE1 board. The segments in this display are called *HEX0[0]*, *HEX0[1]*, . . ., *HEX0[6]*, corresponding to Figure 4. Remember to declare the 7-bit port

```
output reg [0:6] HEX0 // Verilog

HEX0 : OUT STD_LOGIC_VECTOR(0 TO 6); -- VHDL
```

   in your HDL code in such a way that the names of the outputs match the corresponding names in the ***DE1_SoC.qsf*** file.

3. Ask an AI to generate the same functionality you implemented (with the same HDL you employed) and compare the result
4. After adding the required DE1-SoC board **pin assignments**, compile the project.
5. **Download** the compiled circuit in the FPGA chip.
6. **Test** the functionality of the circuit by toggling the *SW*3-0 switches and by looking at the 7-segment display.

1. YOU NEED TO WRITE THE HDL CODE OF THE "HEX" DECODER.
2. YOU NEED TO TEST IT ON THE FPGA.

4 – Comparing two values

A comparator is a simple combinational circuit, which compares two values A and B to state through three flags if A is lower than B (lt), equal to B (eq) or greater than B (gt). Since data can be either signed or unsigned, the correct output of a comparator depends on data format.

Consider the circuit shown in Figure 5. It uses two comparators; each compares A and B and produces three flags: lower than (lt), equal to (eq) and greater than (gt). The first comparator (ucmp) compares A and B as unsigned values, whereas the one (scmp) compares A and B as signed values. The values to be compared are obtained by setting the switches *SW*7-4 for *A* and *SW*3-0 for *B* and are displayed on HEX1 (A) and HEX0 (B). The flags are connected to the LEDs as detailed in Figure 5, namely: ucmp(lt) -> LEDR0, ucmp(eq) -> LEDR1, ucmp(gt) -> LEDR2 and scmp(lt) -> LEDR3, scmp(eq) -> LEDR4, scmp(gt) -> LEDR5.

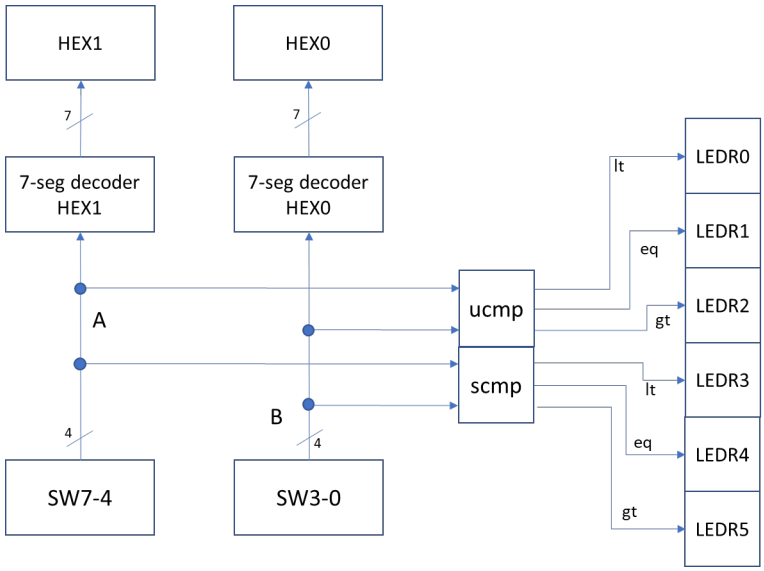


Figure 5 – Comparator-based circuit.

Note: note that both Verilog and VHDL support comparisons with operators <, = and >, but you have to specify if the values you are representing are either signed or unsigned. For the VHDL you need to use the numeric_std packages included in the ieee library.

You need to do the following steps.

1. **Create a new Quartus Prime** project for your circuit.
2. **Create Verilog modules or VHDL entities** for the circuit.
3. Ask an AI to generate the same functionality you implemented (with the same HDL you employed) and compare the result
4. **Include your HDL top-level module/entity** in the Quartus Prime project as well as the other HDL files.
5. Include the required **pin assignments** for the DE1-SoC board for all switches, LEDs and 7-segment displays. Compile the project.
6. **Download** the compiled circuit in the FPGA chip.
7. **Test** the functionality of the circuit by setting the switches, and observe the display and the LEDs.

1. YOU NEED TO WRITE THE HDL CODE OF THE WHOLE CIRCUIT.
2. IN ORDER TO MAKE YOUR HDL CODE AS MUCH REUSABLE AS POSSIBLE YOU CAN MAKE IT PARAMETRIC.
3. YOU NEED TO TEST IT ON THE FPGA.

5 – Comparing and multiplexing

Adding two multiplexers to the previous circuit permits you to make further tests. You can simply connect A and B to the output of two 4-bit, 2-to-1 multiplexers (M_A and M_B respectively). Then, input X of each multiplexer is connected to the switches (X_A to SW7-4 and X_B to SW3-0); the input Y is connected to a constant (Y_A to 8 and Y_B to 13) and the selectors are connected to switches too (s_A to SW9 and s_B to SW8).

1. **Create a new Quartus Prime** project for your circuit.
2. **Modify the top module/entity to include the multiplexers**.
3. Ask an AI to generate the same functionality you implemented (with the same HDL you employed) and compare the result
4. **Check that your HDL top-level module/entity is included** in the Quartus Prime project as well as the other HDL files.
5. Include the required **pin assignments** for the DE1-SoC board for all switches, LEDs and 7-segment displays. Compile the project.
6. **Download** the compiled circuit in the FPGA chip.
7. **Test** the functionality of the circuit by setting the switches, and observe the display and the LEDs.

1. REMEMBER TO TEST THE PROJECT ON THE FPGA.